Java 25 Features for Patterns

Java 25 Features · Model Payment ADT without illegal states · Build with Waqar

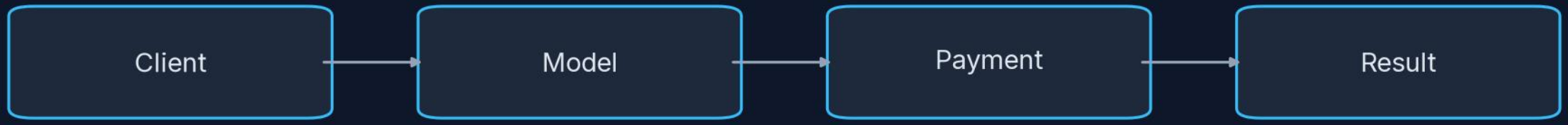
Today's Hook

- Problem: Model Payment ADT without illegal states
- Pattern: Java 25 Features — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Model Payment ADT without illegal states
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Java 25 Features: Model Payment ADT without illegal states

Real-World Scenario

- Running example: Model Payment ADT without illegal states
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.adt;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== ADT / Smart Constructor walkthrough ===");
    System.out.println("email=" + Email.parse("waqar@bwq.dev"));
    System.out.println("badEmail=" + Email.parse("nope"));
    System.out.println(Payment.describe(new CashPayment(100)));
    System.out.println(Payment.describe(new CardPayment(500, "1234")));
    System.out.println(Payment.describe(new WirePayment(900, "IN00")));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.adt;

/** Algebraic data type via sealed records + pattern matching. */
public sealed interface Payment permits CardPayment, CashPayment, WirePayment {
    int amountCents();

    static String describe(Payment p) {
        return switch (p) {
            case CardPayment(var cents, var last4) -> "card ****" + last4 + " " + cents;
            case CashPayment(var cents) -> "cash " + cents;
            case WirePayment(var cents, var iban) -> "wire " + iban + " " + cents;
        };
    }
}

record CardPayment(int amountCents, String last4) implements Payment {}
record CashPayment(int amountCents) implements Payment {}
record WirePayment(int amountCents, String iban) implements Payment {}


package com.buildwithwaqar.patterns.functional.adt;
```

Solution (2)

```java
import dev.zazr.control.Either;

/** Smart constructor — prefer {@link #parse}; canonical ctor still validates. */
public record Email(String value) {
    public Email {
        if (value == null || !value.contains("@")) {
            throw new IllegalArgumentException("invalid email");
        }
    }

    public static Either<String, Email> parse(String raw) {
        if (raw == null || !raw.contains("@")) return Either.left("invalid email");
        return Either.right(new Email(raw));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.adt;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== ADT / Smart Constructor walkthrough ===");
    System.out.println("email=" + Email.parse("waqar@bwq.dev"));
    System.out.println("badEmail=" + Email.parse("nope"));
    System.out.println(Payment.describe(new CashPayment(100)));
    System.out.println(Payment.describe(new CardPayment(500, "1234")));
    System.out.println(Payment.describe(new WirePayment(900, "IN00")));
  }
}
```

Walkthrough Output

```
=== ADT / Smart Constructor walkthrough ===
email=Right(Email[value=waqar@bwq.dev])
badEmail=Left(invalid email)
cash 100
card ****1234 500
wire IN00 900
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Java 25 Features.

Related Patterns

- Contrast Java 25 Features with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Java 25 Features → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.